Algoritmo parallelo di Smith-Waterman per allineamento di sequenze DNA

Christian Pozzoli 29678A

Ottobre 2025



Indice

1 Introduzione **3**
1.1 Allineamento di sequenze di DNA 3
1.2 Obiettivo del progetto 3
1.3 Strumenti e ambiente di sviluppo 4

2 Algoritmo **4**
2.1 Algoritmo di Smith-Waterman 4
2.2 Implementazione sequenziale 5

3 Implementazione su GPU **6**
3.1 Parallelizzazione dell'algoritmo 6
3.2 Implementazione su GPU 6
3.3 Ottimizzazione con blocchi 6
3.4 Ottimizzazione con memoria condivisa (SMEM) 6
3.5 Ottimizzazione con warp e coalescenza 6
3.6 Analisi delle prestazioni e comparazione 6

4 Conclusione **6**

1 Introduzione

1.1 Allineamento di sequenze di DNA

L'allineamento di sequenze di DNA è una delle operazioni fondamentali della bioinformatica e rappresenta uno strumento essenziale per lo studio comparativo dei genomi. L'obiettivo principale dell'allineamento è confrontare due o più sequenze di basi di nucleotidi al fine di individuare delle regioni di somiglianza che possono riflettere relazioni evolutive, funzionali o strutturali tra gli organismi.

In termini pratici, un allineamento consiste nel disporre le sequenze una sopra l'altra, introducendo eventuali spazi vuoti ("gap") per massimizzare il numero di corrispondenze tra le basi. Questa operazione consente di identificare mutazioni, inserzioni o delezioni, e di dedurre l'origine comune di geni o proteine. Queste analisi sono fondamentali in vari ambiti applicativi, tra cui

- l'annotazione di nuovi genomi
- la ricerca di omologie tra geni
- la previsione della funzione di proteine sconosciute
- lo studio delle malattie genetiche

Esistono due principali categorie di allineamento: globale e locale. L'allineamento globale, proposto da Needleman e Wunsch (1970), confronta le sequenze nella loro interezza, ed è più adatto quando le sequenze hanno lunghezze simili e un alto grado di somiglianza. L'allineamento locale, introdotto da Smith e Waterman (1981), mira invece a trovare le regioni più simili all'interno di sequenze potenzialmente molto diverse, risultando particolarmente utile in contesti biologici reali, dove solo alcune porzioni delle sequenze mostrano omologia.

1.2 Obiettivo del progetto

L'obiettivo del progetto è implementare e ottimizzare l'algoritmo di Smith-Waterman sfruttando la potenza di calcolo parallelo delle GPU attraverso la piattaforma CUDA. L'algoritmo di Smith-Waterman presenta una complessità computazionale di tipo quadratico rispetto alla lunghezza delle sequenze da confrontare. Questo lo rende poco adatto all'elaborazione di grandi quantità di dati, come quelli generati dalle moderne tecniche di sequenziamento del DNA.

L'obiettivo principale è quindi ridurre i tempi di esecuzione attraverso una parallelizzazione efficiente delle sue operazioni e mantenendo inalterata la precisione dell'algoritmo.

Il progetto mira a dimostrare come l'uso del calcolo parallelo possa rendere praticabile l'utilizzo dell'algoritmo di Smith-Waterman anche in contesti reali e su dataset di grandi dimensioni.

1.3 Strumenti e ambiente di sviluppo

Lo sviluppo del progetto è stato condotto utilizzando il linguaggio CUDA C, sfruttando l'integrazione diretta con l'architettura di calcolo parallelo delle GPU NVIDIA.

L'ambiente di sviluppo è stato configurato su sistema operativo Windows, utilizzando il compilatore `nvcc` in versione 12.9.86, incluso nel CUDA Toolkit 13.0. Il sistema era dotato dei driver NVIDIA in versione 581.57, pienamente compatibili con la versione del toolkit utilizzato.

Per la fase di test e analisi delle prestazioni è stata impiegata una GPU NVIDIA GeForce RTX 3070 (Compute Capability 8.6).

2 Algoritmo

L'algoritmo di Smith-Waterman permette di ottenere un allineamento locale ottimale tramite un approccio di programmazione dinamica. In questa sezione viene descritto il funzionamento dell'algoritmo, la sua formulazione e la relativa implementazione sequenziale, che farà da riferimento per la successiva versione parallela su GPU.

2.1 Algoritmo di Smith-Waterman

L'algoritmo di Smith-Waterman consente di ottenere un allineamento locale ottimale tra due sequenze, identificando le regioni di massima somiglianza. Nell'allineamento vengono considerati tre tipi di operazioni: corrispondenze o sostituzioni, inserzioni e delezioni. Le inserzioni e le delezioni introducono dei "gap" che consentono di allineare correttamente regioni non perfettamente coincidenti tra le sequenze.

L'algoritmo utilizza la tecnica di programmazione dinamica per costruire la matrice di sostituzione in modo da assegnare un punteggio in base ai possibili allineamenti delle sequenze considerate. La soluzione viene poi ricostruita partendo dall'allineamento con il punteggio più alto.

Si articola in quattro fasi principali:

- **Definizione del modello di punteggio e penalità**
  Ogni coppia di basi (o amminoacidi, nel caso di sequenze proteiche) riceve un punteggio in base alla corrispondenza: le corrispondenze esatte vengono premiate con valori positivi, mentre le sostituzioni ottengono punteggi inferiori o negativi. La scelta dello schema di penalità dipende dal tipo di sequenza analizzata e dagli obiettivi dello studio.

- **Inizializzazione della matrice dei punteggi**
  Si costruisce una matrice bidimensionale di dimensioni pari a $(m + 1) \times (n + 1)$, dove m e n rappresentano le lunghezze delle due sequenze. Tutti gli elementi della prima riga e della prima colonna vengono inizializzati a zero. Questa scelta consente all'allineamento di iniziare in qualsiasi posizione delle sequenze e garantisce che i gap iniziali non siano penalizzati.

- **Calcolo dei punteggi (fase di riempimento della matrice)**
  La matrice viene riempita procedendo da sinistra verso destra e dall'alto verso il basso. Per ogni cella si considerano tre possibili punteggi:
  - il punteggio diagonale (corrispondente a match/mismatch),
  - il punteggio proveniente da sinistra (introduzione di un gap nella sequenza verticale),
  - il punteggio proveniente dall'alto (introduzione di un gap nella sequenza orizzontale).

  Il valore della cella viene impostato come il massimo tra questi tre valori e zero, così da eliminare i contributi negativi e limitare l'allineamento a regioni localmente simili.

- **Traceback (ricostruzione dell'allineamento)**
  Una volta completato il riempimento della matrice, si identifica la cella con il punteggio massimo. A partire da essa, si effettua un traceback seguendo la direzione dei punteggi che hanno contribuito al valore corrente (diagonale, sinistra o alto), fino a raggiungere una cella con valore zero. Il percorso così ottenuto rappresenta l'allineamento locale ottimale. È inoltre possibile individuare allineamenti secondari ripetendo il processo di traceback a partire dai punteggi massimi rimanenti.

Grazie a questa formulazione l'algoritmo di Smith-Waterman garantisce la massima accuratezza possibile nell'allineamento locale. Tuttavia, la sua complessità computazionale quadratica in funzione della lunghezza delle sequenze ($O(m \cdot n)$) ne limita l'applicabilità diretta su dataset di grandi dimensioni, rendendo necessarie strategie di ottimizzazione e parallelizzazione.

2.2 Implementazione sequenziale

```
1  void cpu_alignment(
2                  sw_mat* h,
3                  string seq_1,
4                  string seq_2,
5                  int seq_1_length,
6                  int seq_2_length
7                  ) {
8      for(int i = 1; i < seq_1_length; ++i) {
9          for (int j = 1; j < seq_2_length; ++j) {
10             bool match = seq_1[i] == seq_2[j];
11             int d = h[INDEX(i - 1, j - 1, seq_2_length)].value +
12                             (match ? MATCH : MISMATCH);
13             int u = h[INDEX(i - 1, j, seq_2_length)].value + GAP;
14             int l = h[INDEX(i, j - 1, seq_2_length)].value + GAP;
15             int result = max(max(0, d), max(u, l));
16
17             sw_mat* h_current = &h[INDEX(i, j, seq_2_length)];
18             h_current->value = result;
19
20             if (result == 0)    h_current->dir = STOP;
```

```
            else if (result == d)   h_current->dir = DIAG;
            else if (result == u)   h_current->dir = UP;
            else if (result == l)   h_current->dir = LEFT;
        }
    }
}
```

Per questa implementazione sequenziale sono stati scelti i seguenti punteggi e penalità secondo basi empiriche e di pratica comune:

- corrispondenza 3
- differenza -3
- gap -2

La matrice è implementata in un array monodimensionale, per garantire coerenza e coesione con i successivi algoritmi paralleli, e l'accesso avviene attraverso una macro, definita di seguito, che linearizza l'indice a partire da riga, colonna e lunghezza della sequenza disposta lungo le colonne.

```
#define INDEX(row, col, stride) ( (row) * (stride) + (col) )
```

Le due sequenze sono interscambiabili, in quanto l'obiettivo è quello di trovare degli allineamenti locali, quindi ambivalenti, ma nel codice si è scelto, per motivi di test e chiarezza, di rifersi alle sequenze con nomi quali `main_sequence` e `sequence` (oppure come `sequence_1` e `sequence_2`), mantenendo sempre la `main_sequence` (o `sequence_1`, solitamente la stringa più lunga) un carattere per riga e la `sequence` (o `sequence_2`) un carattere per colonna.

La matrice viene, quindi, riempita

3 Implementazione su GPU

3.1 Parallelizzazione dell'algoritmo

3.2 Implementazione su GPU

3.3 Ottimizzazione con blocchi

3.4 Ottimizzazione con memoria condivisa (SMEM)

3.5 Ottimizzazione con warp e coalescenza

3.6 Analisi delle prestazioni e comparazione

4 Conclusione